TABLAS DE DATOS CON JTABLE EN SWING

De un array a un TableModel propio, con el patrón Modelo-Vista-Controlador

MATERIAL DE CLASE · DESARROLLO DE SISTEMAS ORIENTADO A OBJETOS

Esteban Santini · 2026

AGENDA DE LA CLASE

01 · JTABLE: FILAS Y COLUMNAS

Qué es un JTable y cómo se relaciona con su TableModel interno.

02 · TRES FORMAS DE CARGARLO

De un array de datos a DefaultTableModel y, por último, a un AbstractTableModel propio.

03 · FORMATO Y COMPORTAMIENTO

Ancho de columnas, orden, y por qué en este ejemplo la tabla no es editable.

04 · EJEMPLO MVC COMPLETO

Las notas de una materia, filtradas por estado, con modelo, vista y controlador.

JTABLE: FILAS Y COLUMNAS

javax.swing.JTable — mostrar datos en forma de tabla

- Muestra una colección de datos organizados en filas y columnas, como una grilla de hoja de cálculo.
- No guarda los datos él mismo: como JList y JComboBox, delega todo en un modelo interno, en este caso un TableModel.
- Por defecto permite seleccionar una fila, varias filas, o directamente una celda, según setSelectionMode().
- Igual que JList, casi siempre se usa envuelto en un JScrollPane, para desplazarse cuando hay más filas que espacio visible.
- getValueAt(fila, columna) permite leer cualquier celda directamente desde el componente.

COMPONENTES RELACIONADOS

JTable trabaja siempre junto con:

```
TableModel        →  el modelo de datos
JScrollPane       →  para poder scrollear
TableCellRenderer → cómo se dibuja la celda
import javax.swing.*;
import javax.swing.table.*;
```

LA FORMA MÁS SIMPLE: UN ARRAY

- El constructor JTable(Object[][] datos, Object[] columnas) arma una tabla completa en una sola línea.
- Cada fila interna del array es una fila de la tabla; el segundo array son los títulos de columna.
- Es el punto de partida más rápido para probar algo, pero el modelo generado es genérico y limitado.
- No hay ninguna clase de por medio: los datos quedan "sueltos" en el array, sin relación con el resto de la app.

```
// Instanciar con datos fijos
Object[][] datos = {
    {"Ana García", 8, 7, 9},
    {"Bruno Pérez", 5, 4, 6},
    {"Carla Díaz", 6, 7, 6}
};
Object[] columnas =
    {"Alumno", "Parcial 1", "Parcial 2", "Parcial
3"};

JTable tabla = new JTable(datos, columnas);

// Componer (con scroll)
ventana.add(new JScrollPane(tabla));
```

UN PASO MÁS: DefaultTableModel

- Es el modelo mutable que Swing ya trae resuelto: permite agregar y quitar filas después de crear la tabla.
- addRow(Object[]) suma una fila nueva; removeRow(indice) la quita; setValueAt() modifica una celda puntual.
- La tabla se construye una sola vez, apuntando siempre al mismo modelo, igual que vimos con DefaultListModel.
- Sigue siendo genérico: cada celda es un Object suelto, sin relación con una clase propia del dominio.

```
// Instanciar el modelo dinámico
DefaultTableModel modelo = new DefaultTableModel(
    new Object[]{"Alumno", "Promedio"}, 0);

// Agregar filas en tiempo de ejecución
modelo.addRow(new Object[]{"Ana García", 8.0});
modelo.addRow(new Object[]{"Bruno Pérez", 5.0});

// Componer
JTable tabla = new JTable(modelo);
ventana.add(new JScrollPane(tabla));

// Quitar una fila
modelo.removeRow(1);
```

EL MÁXIMO CONTROL: AbstractTableModel

- Permite que la tabla lea directamente objetos del dominio (por ejemplo, una lista de Alumno), sin duplicar los datos en un array genérico.
- Solo exige implementar tres métodos: getRowCount(), getColumnCount() y getValueAt(fila, columna).
- getColumnName() es opcional, pero conviene sobrescribirlo para no mostrar "A", "B", "C"... como títulos.
- Es la opción más alineada con MVC: el modelo de la tabla puede consultar al modelo de datos real de la aplicación.

```
public class MiTableModel
        extends AbstractTableModel {

    @Override
    public int getRowCount() {
        // cantidad de filas
    }

    @Override
    public int getColumnCount() {
        // cantidad de columnas
    }

    @Override
    public Object getValueAt(
            int fila, int columna) {
        // el valor de esa celda
    }
}
```

TRES FORMAS, UNA MISMA TABLA

Las tres construyen exactamente el mismo JTable en pantalla; lo que cambia es cuánto control (y cuánto código) hace falta.

ARRAY

La más rápida de escribir: una sola línea.
Sirve para pruebas rápidas o demos chicos.
No permite agregar filas después, ni conectar con el resto de la aplicación.

DEFAULTTABLEMODEL

Ya permite agregar y quitar filas en tiempo de ejecución.
Sigue usando Object genérico en cada celda.
No hay una clase propia del dominio detrás de cada fila.

ABSTRACTTABLEMODEL

Más código para escribir al principio.
El modelo puede leer directamente objetos del dominio (como Alumno).
Es la opción recomendada para una aplicación con arquitectura MVC.

CONFIGURAR EL JTABLE

- setAutoResizeMode() controla cómo se reparten las columnas cuando la ventana cambia de tamaño.
- getColumnModel().getColumn(i).setPreferredWidth() ajusta el ancho de una columna puntual.
- setAutoCreateRowSorter(true) habilita ordenar la tabla haciendo clic en cada encabezado, sin escribir ningún comparador.
- setSelectionMode() funciona igual que en JList: selección simple o distintos modos de selección múltiple.

```
// Ancho de columnas
tabla.getColumnModel().getColumn(0)
     .setPreferredWidth(160);

// Repartir el ancho sobrante
tabla.setAutoResizeMode(
    JTable.AUTO_RESIZE_ALL_COLUMNS);

// Permitir ordenar por columna
tabla.setAutoCreateRowSorter(true);

// Selección de una sola fila
tabla.setSelectionMode(
    ListSelectionModel.SINGLE_SELECTION);
```

¿SE PUEDE EDITAR UNA CELDA?

isCellEditable(): quién decide

- Por defecto, un JTable construido sobre DefaultTableModel permite editar cualquier celda con doble clic.
- isCellEditable(fila, columna) es el método que decide, celda por celda, si se puede editar o no.
- Al extender AbstractTableModel, isCellEditable() devuelve false por defecto: hay que sobrescribirlo a propósito para habilitar la edición.
- En el ejemplo de esta clase la tabla es de solo lectura: alcanza con no sobrescribir el método y dejar el comportamiento por defecto.

SI HICIERA FALTA EDITAR

1
Sobrescribir isCellEditable() devolviendo true para las celdas editables.

2
Sobrescribir setValueAt() para guardar el nuevo valor en el objeto real.

3
Llamar a fireTableCellUpdated() para avisarle a la tabla que se repinte.

EL MISMO PATRÓN: COMBO → TABLA

Ya lo vimos con JList — ahora con JTable

- Es exactamente el mismo patrón combo → lista de la clase anterior, aplicado a una tabla en vez de a un JList.
- Se registra un ActionListener sobre el JComboBox de filtro; el JTable no escucha nada directamente.
- La clave sigue siendo no reemplazar el JTable: se actualiza el TableModel que ya tiene asignado y se avisa con fireTableDataChanged().
- Cambia el componente visual (JList → JTable), pero la arquitectura del controlador es idéntica a la que ya conocés.

PASOS DEL PATRÓN

1
El usuario elige un filtro en el combo (Todos / Aprobados / Desaprobados).

2
Se dispara el ActionListener registrado sobre el combo.

3
El controlador filtra la lista de alumnos y avisa al TableModel.

EL EJEMPLO: NOTAS DE UNA MATERIA

Un JComboBox que filtra, una tabla que muestra

- El combo permite elegir qué alumnos ver: todos, solo los aprobados o solo los desaprobados.
- La tabla muestra, para cada alumno, sus tres notas parciales, el promedio calculado y su estado.
- El promedio y el estado no se guardan: se calculan siempre a partir de las tres notas, cada vez que se piden.
- Lo vamos a construir con cuatro clases: Alumno, CalificacionesModelo, AlumnosTableModel y VistaAlumnos + CalificacionesControlador.

ESTRUCTURA DE DATOS

Cada fila de la tabla es un:

```
Alumno { nombre, nota1, nota2, nota3 }
getPromedio() = (n1+n2+n3) / 3.0
estaAprobado() = promedio >= 6.0
```

ARQUITECTURA MVC APLICADA

Aplicamos otra vez el mismo esquema Modelo–Vista–Controlador, ahora con un JComboBox de filtro y un JTable como parte de la vista.



- Regla de oro (la misma de siempre): el modelo nunca importa javax.swing.
- El AlumnosTableModel es parte de la vista: traduce objetos Alumno al formato de filas y columnas que JTable necesita.
- Vamos a construir las cuatro clases en las próximas diapositivas.

1) EL MODELO — LA CLASE Alumno

- Un POJO común y corriente: guarda las tres notas parciales de un alumno, sin ningún import de javax.swing.
- getPromedio() calcula el promedio a partir de las tres notas, cada vez que se le pide: nunca se guarda un valor viejo.
- estaAprobado() reutiliza getPromedio() para decidir el estado: la regla de negocio vive en un solo lugar.
- Va a ser el tipo de dato que viaje entre el modelo, el TableModel y, en definitiva, cada fila de la tabla.

```
// Alumno: un objeto de dominio simple, sin Swing
public class Alumno {

    private final String nombre;
    private final int nota1, nota2, nota3;

    public Alumno(String nombre, int n1, int n2, int n3) {
        this.nombre = nombre;
        this.nota1 = n1; this.nota2 = n2; this.nota3 = n3;
    }

    public String getNombre() { return nombre; }
    public int getNota1() { return nota1; }
    public int getNota2() { return nota2; }
    public int getNota3() { return nota3; }

    public double getPromedio() {
        return (nota1 + nota2 + nota3) / 3.0;
    }

    public boolean estaAprobado() {
        return getPromedio() >= 6.0;
    }
}
```

1) EL MODELO — CalificacionesModelo

- Guarda la lista completa de alumnos y no conoce ningún "filtro activo": cada consulta es independiente.
- obtenerAlumnos(filtro) devuelve siempre una lista nueva, ya filtrada, lista para mostrar.
- El filtro llega como String desde el combo; un switch decide qué alumnos pasan.
- Ni Alumno ni CalificacionesModelo importan javax.swing: se pueden probar con JUnit sin abrir ninguna ventana.

```
// MODELO: datos y lógica de filtrado, sin Swing
public class CalificacionesModelo {

    private final List<Alumno> alumnos =
        new ArrayList<>();

    public CalificacionesModelo() {
        alumnos.add(new Alumno(
            "Ana García", 8, 7, 9));
        alumnos.add(new Alumno(
            "Bruno Pérez", 5, 4, 6));
        alumnos.add(new Alumno(
            "Carla Díaz", 6, 7, 6));
        alumnos.add(new Alumno(
            "Diego Fernández", 3, 5, 4));
        alumnos.add(new Alumno(
            "Elena Suárez", 9, 10, 8));
    }

    public List<Alumno> obtenerAlumnos(String filtro) {
        return alumnos.stream()
            .filter(a -> switch (filtro) {
                case "Aprobados" -> a.estaAprobado();
                case "Desaprobados" -> !a.estaAprobado();
                default -> true;
            })
            .toList();
    }
}
```

2) EL TABLEMODEL: AlumnosTableModel

- Extiende AbstractTableModel y guarda su propia referencia a la lista de alumnos a mostrar.
- getRowCount() y getColumnCount() son casi triviales; todo el trabajo real está en getValueAt().
- getValueAt() arma cada celda a partir de los getters de Alumno, incluyendo el promedio formateado y el estado como texto.
- actualizar() reemplaza la lista interna y llama a fireTableDataChanged(): así la tabla se repinta sola.

```
// TABLEMODEL: adapta la lista de alumnos
public class AlumnosTableModel extends AbstractTableModel {

    private static final String[] COLUMNAS = {
        "Alumno", "Parcial 1", "Parcial 2",
        "Parcial 3", "Promedio", "Estado"};
    private List<Alumno> alumnos = new ArrayList<>();

    public void actualizar(List<Alumno> nueva) {
        this.alumnos = nueva;
        fireTableDataChanged();
    }

    public int getRowCount() { return alumnos.size(); }
    public int getColumnCount() { return COLUMNAS.length; }
    public String getColumnName(int col) { return COLUMNAS[col]; }

    @Override
    public Object getValueAt(int fila, int col) {
        Alumno a = alumnos.get(fila);
        return switch (col) {
            case 0 -> a.getNombre();
            case 1 -> a.getNota1();
            case 2 -> a.getNota2();
            case 3 -> a.getNota3();
            case 4 -> String.format("%.2f", a.getPromedio());
            case 5 -> a.estaAprobado() ? "Aprobado" : "Desaprobado";
            default -> null;
        };
    }
}
```

3) LA VISTA: VistaAlumnos

- Extiende JFrame y expone el combo de filtro y el AlumnosTableModel como atributos.
- El JTable se crea directamente sobre el AlumnosTableModel: no hace falta ningún paso intermedio.
- BorderLayout ubica el combo arriba (NORTH) y la tabla, con scroll, en el centro (CENTER).
- No decide qué alumnos mostrar: solo expone getComboFiltro() y getModeloTabla() para que el controlador actúe.

```
// VISTA: solo interfaz, sin lógica de negocio
public class VistaAlumnos extends JFrame {

    private final JComboBox<String> comboFiltro =
        new JComboBox<>(new String[]{
            "Todos", "Aprobados", "Desaprobados"});
    private final AlumnosTableModel modeloTabla =
        new AlumnosTableModel();
    private final JTable tabla =
        new JTable(modeloTabla);

    public VistaAlumnos() {
        super("Calificaciones");
        setLayout(new BorderLayout(10, 10));
        add(comboFiltro, BorderLayout.NORTH);
        add(new JScrollPane(tabla),
            BorderLayout.CENTER);
        setSize(480, 280);
        setLocationRelativeTo(null);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
    }

    public JComboBox<String> getComboFiltro() {
        return comboFiltro;
    }

    public AlumnosTableModel getModeloTabla() {
        return modeloTabla;
    }
}
```

4) EL CONTROLADOR: CalificacionesControlador

```
// CONTROLADOR: conecta la vista con el modelo
public class CalificacionesControlador {

    private final CalificacionesModelo modelo;
    private final VistaAlumnos vista;

    public CalificacionesControlador(CalificacionesModelo modelo, VistaAlumnos
vista) {
        this.modelo = modelo;
        this.vista = vista;
        vista.getComboFiltro().addActionListener(e -> actualizarTabla());
        actualizarTabla();
    }

    private void actualizarTabla() {
        String filtro = (String) vista.getComboFiltro().getSelectedItem();
        vista.getModeloTabla().actualizar(modelo.obtenerAlumnos(filtro));
    }
}
```

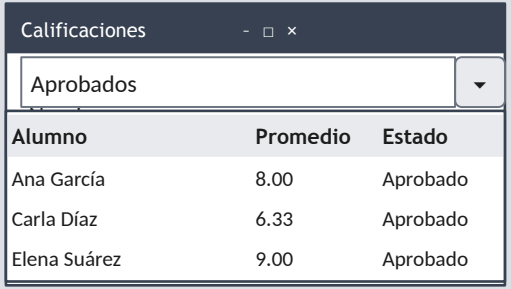
- El único listener registrado es el del combo, igual que en el ejemplo del catálogo.
- actualizarTabla() lee el filtro elegido, se lo pide al modelo y pasa el resultado al TableModel.
- El TableModel es quien decide repintar la tabla: el controlador nunca toca el JTable directamente.
- El main arma las tres piezas y muestra la ventana, como en todos los ejemplos anteriores.

RESULTADO EN EJECUCIÓN

```
// PUESTA EN MARCHA
public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> {
        VistaAlumnos vista = new VistaAlumnos();
        new CalificacionesControlador(
            new CalificacionesModelo(), vista);
        vista.setVisible(true);
    });
}
```

Las cuatro clases ya están armadas: el main solo las instancia y las conecta.
Cada cambio del combo dispara actualizarTabla(), que repuebla la tabla a partir del modelo filtrado.

RESULTADO



BUENAS PRÁCTICAS Y ERRORES COMUNES

BUENAS PRÁCTICAS

- Actualizar el TableModel existente con fireTableDataChanged() en vez de crear un JTable nuevo cada vez.
- Dejar que AbstractTableModel calcule los valores derivados (como el promedio) en getValueAt(), sin guardarlos duplicados.
- Envolver el JTable en un JScrollPane: sin scroll, las tablas largas se cortan igual que las listas.
- Sobrescribir getColumnName() para que los encabezados sean legibles, en vez de dejar "A", "B", "C"...

ERRORES COMUNES

- Olvidar fireTableDataChanged() después de modificar los datos: la tabla no se entera del cambio.
- Guardar el promedio como un campo mutable en Alumno en vez de calcularlo: fácil que quede desactualizado.
- Dejar la tabla editable por accidente, sin sobrescribir isCellEditable(), cuando el diseño pide solo lectura.
- Reconstruir todo el TableModel en cada evento en vez de reutilizar la misma instancia.

El mismo consejo de siempre: la solución casi nunca es tocar el componente Swing, sino actualizar su modelo y dejar que él se encargue de redibujarse.

RESUMEN Y BUENAS PRÁCTICAS

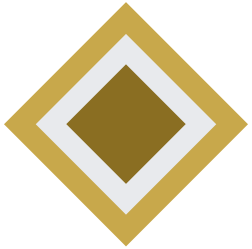
- JTable, igual que JList y JComboBox, separa los datos de la presentación a través de un modelo: en este caso, TableModel.
- Para tablas simples alcanza con un array o un DefaultTableModel; para aplicaciones con MVC, un AbstractTableModel propio es la opción más prolija.
- El patrón combo → vista que ya conocías se aplica igual acá: el controlador filtra en el modelo y actualiza el TableModel.
- La arquitectura no cambia entre ejemplos: modelo sin Swing, vista sin lógica, controlador coordinando.
- El ejemplo completo (Alumno, CalificacionesModelo, AlumnosTableModel, VistaAlumnos, CalificacionesControlador) queda como referencia.

SIDE QUEST

Práctica: agregá una columna más a la tabla.

Una columna "Posición" que muestre el ranking de cada alumno según su promedio, recalculada cada vez que cambia el filtro.

Documentación: docs.oracle.com → How to Use Tables



MUCHAS GRACIAS

"Una tabla no es una hoja de cálculo: es una vista de tus objetos."

Preguntas y consultas: ahora, o por los canales de la materia.
En la próxima clase: práctica integradora con los tres componentes basados en modelo.